

pdf_options:
format: a4
margin: 20mm

```
<div style="text-align: center; margin-top: 150px; margin-bottom: 250px;">  
<h1 style="font-size: 48px; margin-bottom: 20px;">Project Report</h1>  
<h2 style="font-size: 32px; color: #555; margin-bottom: 40px;">Vendor Connect India</h2>  
<h3 style="font-size: 24px; color: #777;">Developed in AntigraVity</h3>  
<br/><br/><br/>  
<p style="font-size: 18px;"><strong>Date:</strong> April 3, 2026</p>  
<p style="font-size: 18px;"><strong>Version:</strong> 1.0.0</p>  
</div>
```

```
<div style="page-break-after: always;"></div>
```

Table of Contents

- 1. [Executive Summary](#)
- 2. [Project Overview](#)
- 3. [Stakeholders & Team](#)
- 4. [System Architecture](#)
- 5. [Features & Modules](#)
- 6. [Workflow & Process Flow](#)
- 7. [Implementation Details](#)
- 8. [Testing & QA](#)
- 9. [Deployment & Infrastructure](#)
- 10. [Results & Outcomes](#)
- 11. [Future Enhancements](#)
- 12. [Conclusion](#)
- 13. [Appendix](#)

```
<div style="page-break-after: always;"></div>
```

1. Executive Summary

Vendor Connect India (also known as the Street Vendor Digital Shop Builder) is a comprehensive web platform designed to empower local street vendors by enabling them to create a digital presence. Built natively with the assistance of AntigraVity AI, the project aims to solve the visibility and modern commerce gap for street vendors by offering essential e-commerce capabilities such as digital menus, shop creation, location-based discoverability, and Razorpay integrated digital payments. This report details the successful development, end-to-end testing, and deployment of a secure, responsive, and robust platform bridging vendors and nearby consumers.

2. Project Overview

Background

Street vendors constitute a significant segment of the local economy in India. However, their reliance on foot traffic and traditional word of mouth severely limits customer reach. Furthermore, their inability to quickly accept digital orders sets them at a commercial disadvantage compared to established brick-and-mortar storefronts on large food-delivery networks.

Problem Statement

Local vendors lack an affordable, easy-to-use digital platform to list products, share locations, and receive digital orders and payments securely. Customers equally struggle to find, assess, and interact with reliable local street vendors.

Objectives

- Provide a simple digital onboarding flow for vendors.
- Enable geolocation-based discovery so local consumers can find shops within a 5km radius.
- Facilitate digital menu management and ratings.
- Enable secure payment processing capabilities.

Scope

The scope of Version 1.0.0 encompasses vendor registration, authentication, dynamic shop creation, menu updates, real-time map integration, and reviews. The application handles robust security (JWT, rate-limiting, Helmet.js headers) and utilizes Razorpay for checkout flows.

```
<div style="page-break-after: always;"></div>
```

3. Stakeholders & Team

- **Vendors:** Primary users. Can register, set up their shop profile, manage menus, list prices, and process payments.
- **Buyers / Consumers:** Can view maps, locate vendors using Haversine-based radius matching, view menus, and leave reviews (1-5 ratings).
- **Admins & Support:** Can oversee vendor activities, intervene if issues arise, and monitor platform health.
- **Development Team:** Responsible for the MERN stack implementation, UX/UI improvements, testing, deployment, and security auditing in collaboration with AntigraVity AI.

4. System Architecture

The project utilizes the MERN stack (MongoDB, Express, React, Node.js) configured for optimal security and performance.

Tech Stack

- **Frontend:** React 19, Vite, TailwindCSS for styling, React Router for navigation, Leaflet/React-Leaflet for mapping.
- **Backend:** Node.js >=18, Express.js for REST APIs.
- **Database:** MongoDB with Mongoose ODM.

- **Integrations:**
 - **Cloudinary:** For secure handling and storage of vendor shop and product images using `multer-storage-cloudinary`.
 - **Razorpay:** For processing checkout flows and managing digital payments securely.
 - **Leaflet:** Map rendering.

Architecture Diagram (Descriptive)

1. **Client Layer:** A responsive React SPA hosted on Vercel. Communicates with backend endpoints via Axios.
2. **API Gateway / Application Layer:** Node.js/Express server hosted on Render. Handles rate limiting, authentication (JWT), and API requests.
3. **Data Layer:** MongoDB schema stores `Users`, `Shops`, `Products/Menus`, and `Reviews`.
4. **Third-Party Services:** Image assets uploaded directly to Cloudinary. Payments securely verified via Razorpay APIs.

<div style="page-break-after: always;"></div>

5. Features & Modules

Vendor Registration & Security

- Secure signup with field-level validation (password strength indicators, min-length requirements).
- Bcrypt password hashing.
- Environment-gated logging to prevent PII exposure in production.

Dynamic Shop & Menu Management

- **Profile Creation:** Vendor enters shop name, category, and precise geolocation.
- **Menu Builder:** Vendors add items, set pricing, and modify availability flags dynamically.

Matchmaking & Live Map (Consumer Facing)

- Map rendering powered by Leaflet. React hooks capture exact browser UI lat/long.
- Server-side Haversine formula determines vendors operating within a 5km radius.

Ratings & Reviews

- Buyers submit 1-5 star ratings with textual feedback. Average metrics auto-calculate per shop.

Digital Payments

- Users trigger Razorpay checkout when placing orders. Backend cryptographic signature verification ensures transaction validity before updating application state.

<div style="page-break-after: always;"></div>

6. Workflow & Process Flow

1. **Onboarding:** A vendor browses to the platform, uses the sign-up form, and successfully logs in. JWT token persists locally.
2. **Setup:** Vendor prompts the location picker, adding their stall's exact location, shop category, and image.
3. **Menu Addition:** Vendor builds their digital storefront inventory.
4. **Discovery (Customer):** A customer loads the Live Map. The browser shares coordinates, passing them to the backend which filters the vendor database within 5km.
5. **Purchase & Review:** The customer selects the vendor, adds items to their cart, pays via Razorpay, and optionally leaves a 1-5 rating on the vendor.

7. Implementation Details

Development Approach

The project followed an agile methodology broken into six primary phases: Environment configuration, Authentication, Core Features, Error Handling, API Connections, and Final Polish.

Key Tools Used

- `express-rate-limit` and `helmet` for production security.
- `framer-motion` and `lucide-react` for aesthetic UI interactions.
- `dotenv` for environment separation.

Challenges Faced

- **CORS and API Routing:** Fixed severe misconfigurations where the frontend was hardcoded to `localhost` failing on deployment.
- **Security Vulnerabilities:** Addressed brute-force vulnerabilities by introducing standard rate-limiting controls over `/api/auth` routes.
- **UX Validation:** Solved UI issues by upgrading from generic browser alerts to interactive, real-time validation visual cards.
- **UI Responsiveness:** Overcame mobile responsive issues with map filter logic and shop name constraints on smaller screens.

<div style="page-break-after: always;"></div>

8. Testing & QA

A comprehensive end-to-end security and UX audit was performed in Antigravity.

Strategy

- **Unit and Integration Testing:** API testing with complete Postman collections and environments.
- **Security Validations:** Focused testing on API limits and token expiration workflows.

Bug Resolution Summary

- **Critical Fixes:** Real-time form validations established to prevent database 400 errors. Rate limits applied. Hardcoded `localhost` values resolved for production paths. Corrected Haversine mapping calculations.

- Over 6 critical security vulnerabilities and 7 critical UX front-end bugs resolved seamlessly, achieving a true production-ready state.

9. Deployment & Infrastructure

Environments

- **Local / Dev:** Hosted via Vite (Frontend) and Nodemon (Backend).
- **Production:**
 - **Frontend Deploy:** Vercel. Configured with environment variables to proxy to Render.
 - **Backend Server:** Render. Environment variables `NODE_ENV=production` configured.

CI/CD

Git-based push-to-deploy strategy enabled across Vercel and Render for simple update rollouts. Continuous deployment enabled on main branch.

<div style="page-break-after: always;"></div>

10. Results & Outcomes

- **Successful Delivery:** The application meets its core objective by enabling seamless shop creation by vendors and map-based exploration by consumers.
- **Security:** The platform is hardened against typical web vulnerabilities.
- **Business Value:** Allows street vendors to break into the digital space affordably with a UI equivalent to large startup tech platforms.

11. Future Enhancements

The platform is designed with scalability in mind.

- **Email/SMS Verification:** Confirming Vendor integrity at registration.
- **Analytics Dashboard:** Granular views for vendors showing their daily popularity and sales.
- **Logistics Integration:** Potential integration with local delivery agents.
- **Systematic Error Tracking:** Implementing Sentry or DataDog for higher tier logging.

12. Conclusion

Vendor Connect India represents a significantly robust implementation using modern MERN stack architecture. Under Antigravity's accelerated deployment workflows, a highly refined, secure, and performant digital ecosystem was established. From complex Haversine geometric filtering to encrypted Razorpay payment handoffs, the application delivers robust business value reliably across devices.

<div style="page-break-after: always;"></div>

13. Appendix

Glossary

- **JWT:** JSON Web Token
- **Haversine Formula:** An equation giving great-circle distances between two points on a sphere from their longitudes and latitudes.
- **CORS:** Cross-Origin Resource Sharing.
- **Mongoose ODM:** Object Data Modeling library for MongoDB and Node.js.

References

- **Project Structure:** Internal Codebase, Postman Collections, & Security Audits (`AUDIT_REPORT.md` / `CRITICAL_BUG_FIX.md` / `TEST_LOG.md`)
- **Node.js:** Express Security Best Practices Documentation.
- **Frontend:** React Leaflet and Tailwind CSS Official Guidelines.